Système de Gestion de Bases de Données

Chapitre 4: Langage SQL sous Oracle

Mme Salma Ben Saïd
Maître Assistant, Département GPI, INSAT
Laboratoire de recherche SITI, ENIT
salma.bensaid@insat.ucar.tn
salma_ben_said@yahoo.fr

Partie 2

Langage de Manipulation des Données (LMD)

- Insertion des données
- Suppression de données
- Mise à jour de données
- Interrogation des données
- Opérations sur le résultat
- Opérations ensemblistes
- Les jointures
  - Sélection multi-tables
  - Auto-jointure
  - Jointure externe
- Le groupement
- Les sous-requêtes
- Les vues

LMD : **Insertion des données**

INSERT INTO ....

Insertion d'une seule ligne à la fois

Syntaxe :

INSERT INTO NomTable (nc1, nc2,..., ncN) VALUES (valeur1, valeur2, ..., valeurN);

Doivent correspondre en nombre, en ordre et en type de données.

L'insertion obéit au principe du tout ou rien.

Schéma relationnel :

On reprend le même schéma relationnel :

Client (numClient, nomClient, prenomClient, dat_naiss, adresse)
Commande (numCommande, datCde, #numClient)
Article (numArticle, designation, couleur, prixHT)
LigneCommande(#numCmde,#numArticle, qteCdee)
Refus (#numCmde,#numArticle, datRefus, qteRefusee, motif)

LMD : **Insertion des données**

INSERT INTO ....

Exemple:

```
INSERT  INTO    Client (numClient, nomClient, prenomClient, dat_naiss, adresse)
 VALUES (1, 'nomClient1', 'pClient1','03/12/1970','Tunis');
-- pas obligatoire de citer les colonnes si elles seront toutes renseignées, respecter leur ordr
INSERT  INTO    Client  VALUES (1, 'nomClient1', 'pClient1','03/12/1970','Tunis');
--  obligatoire de citer les colonnes à renseigner, celles non citées auront les valeurs définie
INSERT  INTO    Client (numClient, nomClient, adresse) VALUES (1, 'nomClient1','Tunis');
```

LMD : **Insertion des données**

INSERT INTO ....

Insertion de plusieurs lignes à la fois

Syntaxe:

```
INSERT INTO NomTable (nc1, nc2,..., ncN)
SELECT  nomcol1, nomcol2, ..., nomcolN
FROM ... ;
```

Doivent correspondre en nombre, en ordre et en type de données.

Exemple:

Insérer dans la table CommandeAncienne (même structure que la table commande), les commandes qui datent de plus de six mois.

```
INSERT INTO CommandeAncienne (numCommande, datCde, numClient)
SELECT numCommande, datCde, numClient
FROM Commande
WHERE SYSDATE – datCde > 6*30 ;
```

La soustraction entre dates génère le résultat en nombre de jours.

LMD : **Suppression de données**

DELETE FROM....

Suppression de lignes de données.

Syntaxe:

```
DELETE FROM NomTable ; -- Vide la table

DELETE FROM NomTable
WHERE conditions de sélection ;      /* Supprime les lignes qui satisfont les conditions de sélection */
```

Exemple:

Supprimer de la table Commande, les commandes qui datent de plus de six mois.

```
DELETE FROM Commande WHERE SYSDATE – datCde > 6*30 ;
```

LMD : **Mise à jour de données**

UPDATE NomTable SET....

Syntaxe:

```
UPDATE  nomTable
SET nC1 = valeur,
    nC2 = valeur,
    .
    .
    nCn = valeur
WHERE   condition(s);
```

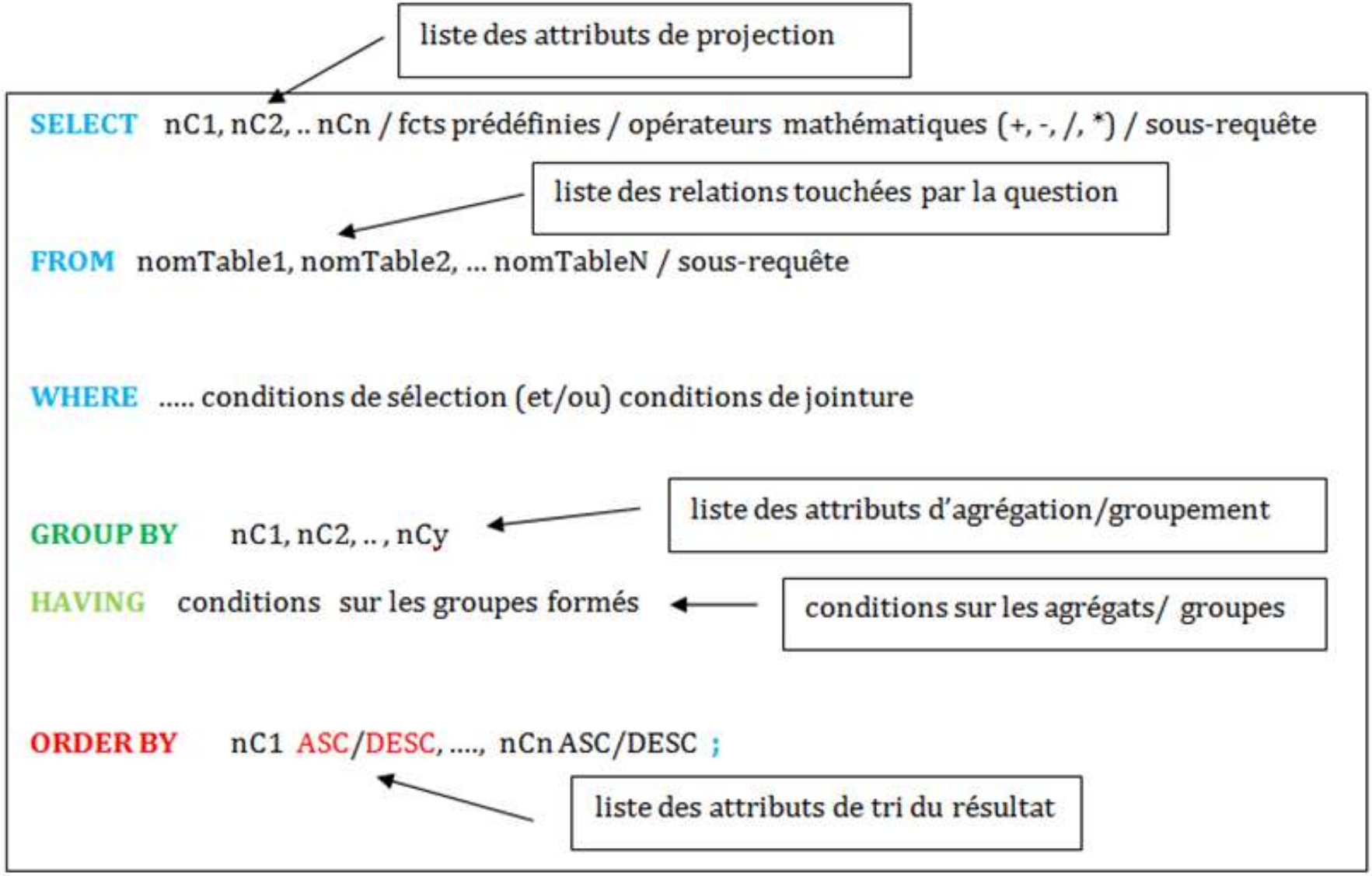
Exemple:

Mettre à jour la couleur à 'bleu' et le prixHT à 130.50 de l'article numéro 120.

```
UPDATE Article
SET couleur = 'bleu' ,
      prixHT = 130.5
WHERE numArticle = 120;
```

LMD : **Interrogation des données** :

La requête SELECT ... FROM ...



LMD : **Interrogation des données** :

La clause : SELECT

Exemple:

Afficher la liste de tous les clients.

Le * remplace toutes les colonnes

```
SELECT *
FROM Client;
```

Exemple:

Afficher nom et prénom des clients qui habitent à Tunis.

```
SELECT nomClient, prenomClient
FROM Client
WHERE UPPER(adresse) = 'TUNIS';
```

LMD : **Interrogation des données** :

La clause : SELECT

Colonnes calculées:

Au niveau de la clause SELECT, on peut faire appel à des fonctions prédéfinies, effectuer des calculs etc.

Exemple:

Afficher l'identité (initiale + nom du client) et l'âge des clients .

```
SELECT SUBSTR(prenomClient,1,1) ||'.'|| nomClient AS "identité", FLOOR((SYSDATE-dat_Naiss)/365) AS "âge"

FROM Client ;
```

On définit des alias pour les noms des colonnes affichées

SUBSTR(chaine, p,n): extrait d'une chaine, à une position p donnée, n caractères.
FLOOR(A/B) : division entière de A par B.

LMD : **Interrogation des données** :

La clause : SELECT

Colonnes calculées:

Exemple :

Calculer et afficher les prix TTC de tous les articles, sachant que la TVA est de 18%.

```
SELECT numArticle, prixHT, prixHT*1.18 AS "prixTTC"

FROM Article ;
```

Fonctions d'agrégations :

Fonction	Description
COUNT	Dénombrement de lignes
MIN	Valeur numérique minimale
MAX	Valeur numérique maximale
AVG	Moyenne des valeurs numériques
SUM	Somme des valeurs numériques

LMD : **Interrogation des données** :

La clause : SELECT

Fonctions d'agrégations :

Elles ignorent la valeur NULL et ne sont pas autorisées directement devant la clause WHERE.

Exemple :

Calculer et afficher le prix de l'article le plus cher, le moins cher et le prix moyen.

```
SELECT MAX(prixHT) AS "prix le + cher", MIN(prixHT) AS "prix le - cher",
 AVG(prixHT) AS "prix moyen"
FROM Article ;
```

LMD : **Interrogation des données** :

La clause : WHERE

Ecriture des conditions

Au niveau de la clase WHERE, on exprime les conditions de sélection, en utilisant les opérateurs de comparaison usuels, spécifiques et les opérateurs logiques.

Opérateurs de comparaison

- = : égal
- <> : différent
- > : plus grand que
- >= : plus grand ou égal
- < : plus petit que
- <= : plus petit ou égal

Opérateurs Logiques

- AND : Tous les prédicats doivent être vérifiés.
- OR : Il suffit que l'un des prédicats soit vérifié.
- NOT : Négation de la condition .

LMD : **Interrogation des données** :

La clause : WHERE

Ecriture des conditions

Opérateurs : **IN, BETWEEN, LIKE**

- Appartenance/Non appartenance à un ensemble :

  IN (…, …., ….)/ NOT IN (…, …., ….)

- Appartenance / Non appartenance à un intervalle :

  BETWEEN a AND b/NOT BETWEEN a AND b :(avec a < b)

- Condition partielle: recherche de motif dans une chaîne :

  LIKE `…% … _ ….` / NOT LIKE `…% … _ ….`

avec: % : remplace n'importe série de chaine de caractères y compris le vide.

_ : remplace exactement un caractère.

LMD : **Interrogation des données** :

La clause : WHERE

Exemple :

Afficher les articles de couleur bleu et dont le prix est compris entre 100 et 300.

```
SELECT *
FROM Article
WHERE couleur ='bleu'
   AND prixHT BETWEEN 100 AND 300;
```

Ou bien

```
SELECT *
FROM Article
WHERE couleur ='bleu'
   AND prixHT >= 100 AND prixHT <= 300;
```

LMD : **Interrogation des données** :

La clause : WHERE

Exemple :

Afficher les clients dont le nom est soit dupond, dubois ou dupret.

```
SELECT *
FROM Client
WHERE  LOWER(nomClient) IN ('dupond','dubois','dupret');
```

Ou bien

```
SELECT *
FROM Client
WHERE  LOWER(nomClient) = 'dupond'
OR LOWER(nomClient) = 'dubois'
OR LOWER(nomClient) = 'dupret';
```

LMD : **Interrogation des données** :

La clause : WHERE

Exemple :

```
SELECT  *
FROM Client
WHERE    prenomClient LIKE    '%s%';      // comporte un s
                              '%s';        //  se termine par un s
                         's%';            // commence par un s
                   's%s';        // commence et se termine par un s
          '__s';        // se termine par un s et comporte exactement 4 caractères
```

LMD : **Interrogation des données** :

La clause : WHERE

Ecriture des conditions

Valeur : **NULL**

- Champs non renseigné/Champs renseigné :

  attribut **IS NULL**/ attribut **IS NOT NULL**

  vide / non vide

Exemple :

```
SELECT *
FROM Client
WHERE adresse IS NULL ; // dont l'adresse n'est pas renseignée
      IS NOT NULL ; // dont l'adresse est renseignée

WHERE adresse = NULL ; // FAUX
```

Opérations sur le résultat

DISTINCT : élimine les doublons lors de l'affichage.

Une requête peut afficher une ligne plusieurs fois, il s'agit de doublons.

Pour éliminer les doublons, on utilise le mot clé DISTINCT juste après le SELECT.

Exemple : afficher la liste des patronymes des clients.

```
SELECT  DISTINCT nomClient
FROM Client  ;
```

ORDER BY ...ASC/DESC : affichage trié.

Exemple :

afficher la liste des clients par ordre décroissant du prénom.

```
SELECT  *
FROM Client
ORDER BY prenomClient DESC ;
```

Exemple :

afficher la liste des clients par ordre croissant du nom et décroissant du prénom.

```
SELECT  *
FROM Client
ORDER BY nomClient ASC, prenomClient DESC ;
```

Opérations sur le résultat

ROWNUM : colonne cachée qui donne le numéro de la ligne affichée.

Chaque requête affichée possède une colonne cachée qui indique le numéro de la ligne affichée .

Exemple : afficher la liste des commandes.

```
SELECT  ROWNUM, Commande.*
FROM Commande  ;
```

ROWNUM	NUMCOMMANDE	DATCDE	NUMCLIENT
1	1	12/20/2019	101
2	2	12/20/2019	102
3	3	12/23/2019	103
4	4	12/29/2019	101
5	5	12/23/2019	103
6	6	12/28/2019	102

Exemple : afficher la liste des Articles.

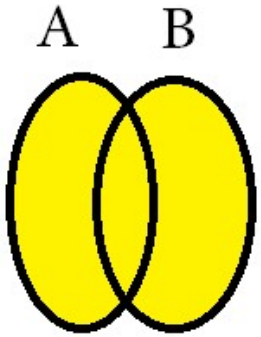
```
SELECT  ROWNUM, Article.*
FROM Article  ;
```

ROWNUM	NUMARTICLE	DESIGNATION	COULEUR	PRIXHT
1	11	Clavier Gamer	Noir	250
2	12	Souris SF	Blanc	70
3	13	Galaxy S50	Silver	3000
4	14	iPhone XS	Gold	4000

Opérations Ensemblistes

UNION (ALL)/ INTERSECT / MINUS.

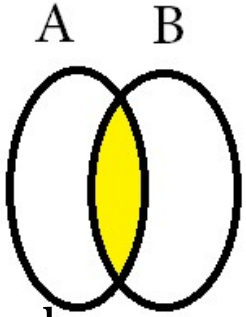
Ensembles de lignes de données homogènes formées par des opérations ensemblistes entre des requêtes dont les colonnes correspondent en nombre, en ordre et en type de données.



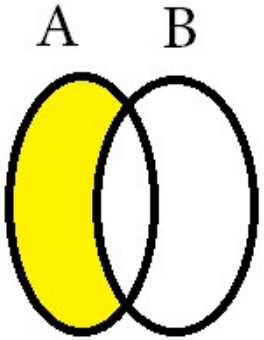
UNION : union de deux ensembles, élimine les doublons, opérateur commutatif.

UNION ALL : conserve les doublons

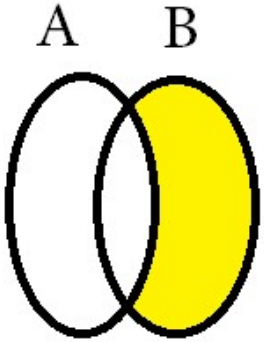
INTERSECT : intersection entre deux ensembles, opérateur commutatif.



MINUS : différence entre deux ensembles, opérateur non commutatif



A **MINUS** B



B **MINUS** A

Opérations Ensemblistes

UNION (ALL)/ INTERSECT / MINUS.

Exemple : Afficher l'ensemble des patronymes des clients et des employés, trié par ordre décroissant du nom.

```
SELECT nomClient AS "nom",prenomClient AS "prenom"
FROM Client
UNION
SELECT nomEmp, prenomEmp
FROM Employe
ORDER BY "nom" DESC;
```

On définit des alias de colonne pour l'affichage au niveau de la première requête.

Opérations Ensemblistes

UNION (ALL)/ INTERSECT / MINUS.

Exemple : Afficher l'ensemble des patronymes communs entre clients et employés, trié par ordre décroissant du nom.

```
SELECT nomClient AS "nom",prenomClient AS "prenom"
FROM Client
INTERSECT
SELECT nomEmp, prenomEmp
FROM Employe
ORDER BY "nom" DESC;
```

Opérations Ensemblistes

UNION (ALL)/ INTERSECT / MINUS.

Exemple : Afficher l'ensemble des patronymes des clients qui n'existent pas dans employés, trié par ordre décroissant du nom.

```
SELECT nomClient AS "nom",prenomClient AS "prenom"
FROM Client
MINUS
SELECT nomEmp, prenomEmp
FROM Employe
ORDER BY "nom" DESC;
```

Opérations Ensemblistes

UNION (ALL)/ INTERSECT / MINUS.

Exemple : Afficher l'ensemble des patronymes des employés qui n'existent pas dans Clients, trié par ordre décroissant du nom.

```
SELECT nomEmp AS "nom", prenomEmp AS "prenom"
FROM Employe
MINUS
SELECT nomClient ,prenomClient
FROM Client
ORDER BY "nom" DESC;
```

LMD : **Les jointures**

Sélection multi-tables

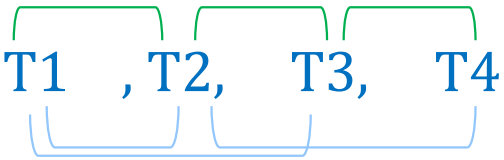
Syntaxe:

```
SELECT .....
FROM  NomTable1, ....., NomTableN
WHERE  NomTable1.ncolA = NomTable2.ncolB
AND ..... ;
```

condition(s) de jointure ⇔ égalités entre colonnes

N tables donc au minimum N-1 conditions de jointure.

..... FROM T1 , T2, T3, T4



Les tables doivent être reliées deux à deux entre elles.
Sinon une opération de multiplication sera réalisée.

Schéma relationnel :

On reprend le même schéma relationnel :

Client (numClient, nomClient, prenomClient, dat_naiss, adresse)
Commande (numCommande, datCde, #numClient)
Article (numArticle, designation, couleur, prixHT)
LigneCommande(#numCmde,#numArticle, qteCdee)
Refus (#numCmde,#numArticle, datRefus, qteRefusee, motif)

LMD : **Les jointures**

Jointure interne

Exemple :

Afficher les clients qui ont passé des commandes.

```
SELECT a.*
FROM Client a, Commande b
WHERE  a.numClient= b.numClient ;
```

Ecriture de la requête avec l'opérateur JOIN

```
SELECT a.*
FROM Client a  INNER JOIN  Commande b
ON ( a.numClient= b.numClient ) ;
```

LMD : **Les jointures**

Jointure interne

Exemple :

Afficher les articles de la commande numéro 1.

```
SELECT a.*
FROM Article a, LigneCommande b
WHERE  a.numArticle= b.numArticle
AND b.numCmde =1;
```

Ecriture de la requête avec l'opérateur JOIN

```
SELECT a.*

FROM Article a INNER JOIN LigneCommande b

 ON ( a.numArticle= b.numArticle )

WHERE  b.numCmde =1;
```

LMD : **Les jointures**

Jointure interne

```
--Afficher les clients qui ont acheté des
--quantités par article supérieures à 5 (qté>5)
SELECT DISTINCT b.*
FROM ligneCommande a, Client b, Commande c
WHERE c.numClient=b.numClient
AND a.numcmde=c.numcommande
AND a.qtecdee >5 ;
```

Ecriture de la requête avec l'opérateur JOIN

```
SELECT DISTINCT b.*
FROM  Client b INNER JOIN Commande c ON( c.numClient=b.numClient )
INNER JOIN  ligneCommande a  ON( a.numcmde=c.numcommande)
WHERE a.qtecdee >5 ;
```

Après chaque opérateur INNER JOIN, définir les conditions de jointures reliant les tables concernées.

LMD : **Les jointures**

Jointure interne

```
SELECT DISTINCT b.*
FROM  Client b INNER JOIN Commande c
INNER JOIN ligneCommande a
ON( a.numcmde=c.numcommande AND c.numClient=b.numClient)
WHERE a.qtecdee >5 ;
```

Résultat de requête

SQL | Exécution :SELECT DISTINCT b.*FROM Client b INNER JOIN Commande c INNER JOIN ligneCommande a ON(a.numcmde=c.numcomm

```
ORA-00905: mot-clé absent
00905. 00000 - "missing keyword"
*Cause:
*Action:
Erreur à la ligne 16, colonne 1
```

Les conditions de jointures impliquant des tables de différents opérateur JOIN ne peuvent pas être jumelées.

LMD : **Les jointures**

Jointure interne

```
-- Afficher pour chaque article retourné, son numéro
-- le n° de la commande correspondante, la qté commandée,
-- la qté refusée et le n° du client concerné.
SELECT a.numArticle,a.numcmde, b.qtecdee, a.qterefusee, c.numclient
FROM  Refus  a INNER JOIN  ligneCommande b
ON( a.numcmde=b.numcmde AND a.numArticle=b.numArticle )
INNER JOIN Commande c  ON( b.numcmde=c.numcommande);
```

Avec l'opérateur NATURAL JOIN on ne peut pas utiliser les alias de tables.

```
SELECT numArticle,numcmde, qtecdee, qterefusee, numclient
FROM  Refus NATURAL JOIN  ligneCommande
INNER JOIN Commande  ON( numcmde=numcommande);
```

LMD : **Les jointures**

Auto-jointure

Syntaxe:

```
SELECT .....
FROM    NomTable1   A , NomTable1   B
WHERE  A.ncol = B.ncol ...;
```

La définition des alias des tables est obligatoire pour pouvoir différencier entre elles.

L'auto-jointure est utilisée lorsqu'on est amené à comparer entre les valeurs d'une même colonne d'une même table.

Auto-jointure

Exemples :

Afficher les couples de numéros des clients qui ont passé des commandes à la même date et afficher cette date.

```
SELECT a.numClient, b.numClient, a.datCde
FROM Commande a, Commande b
WHERE  a.datCde= b.datCde
AND a.numCommande < b.numCommande;
```

Afficher les couples de noms des clients qui ont passé des commandes à la même date et afficher cette date.

```
SELECT c.nomClient, d.nomClient, a.datCde
FROM Commande a, Commande b , Client c,  Client d
WHERE  a.datCde= b.datCde
AND a.numCommande < b.numCommande
AND a.numClient = c.numClient
AND b.numClient = d.numClient;
```

LMD : **Les jointures**

Jointure externe

Syntaxe:

```
SELECT .....
FROM    NomTable1   A, NomTable2    B
WHERE  A.ncolx = B.ncoly (+) ... ;
```

(+) : Opérateur de jointure externe.

La jointure externe indique que l'égalité entre ces colonnes n'est pas obligatoire. Égalités entre clé primaire et clé étrangère.

Le (+) est placé au niveau de l'égalité du côté de la table dont la valeur peut être Null par rapport à sa correspondante dans l'autre table.

S'il y a plusieurs jointures au niveau de la clause WHERE, le (+) doit être placé à chaque égalité.

LMD : **Les jointures**

Jointure externe

Exemple :

Afficher la liste de tous les clients et les détails de leurs commandes éventuelles respectives.

```
SELECT a.numClient,a.nomClient, b.numCommande, c.numArticle, c.designation, d.qteCdee

FROM Client a, Commande b, Article c, LigneCommande d

WHERE  a.numClient= b.numClient(+)

AND b.numCommande = d.numCmde(+)

AND c.numArticle(+)= d.numArticle;
```

LMD : **Les jointures**

Jointure externe

Exemple :

Afficher la liste de tous les clients et les détails de leurs commandes éventuelles respectives (y compris qteRefusee).

```
SELECT a.numClient, a.nomClient, b.numCommande, d.numArticle, c.designation,
d.qteCdee, prixHT*1.18 AS prix_TTC, e.qteRefusee
FROM Client a, Commande b, Article c, LigneCommande d , Refus e
WHERE a.numClient = b.numClient(+)
AND b.numCommande = d.numCmde(+)
AND d.numArticle = c.numArticle(+)
AND d.numCmde = e.numCmde(+)
AND d.numArticle = e.numArticle(+)
ORDER BY a.numClient ASC;
```

LMD : **Les jointures**

Jointure externe

Avec l'opérateur OUTER JOIN :

```
SELECT a.numClient, a.nomClient, b.numCommande, d.numArticle, c.designation,
d.qteCdee, prixHT*1.18 AS prix_TTC, qteRefusee
FROM Commande b RIGHT OUTER JOIN Client a ON( a.numClient = b.numClient)
LEFT OUTER JOIN  LigneCommande d  ON(b.numCommande = d.numCmde)
LEFT OUTER JOIN  Article c ON(d.numArticle = c.numArticle)
LEFT OUTER JOIN Refus e  ON(d.numCmde = e.numCmde
AND d.numArticle = e.numArticle);
--ou
SELECT a.numClient, a.nomClient, b.numCommande, d.numArticle, c.designation,
d.qteCdee, prixHT*1.18 AS prix_TTC , qteRefusee
FROM Client a LEFT OUTER JOIN Commande b  ON( a.numClient = b.numClient)
LEFT OUTER JOIN  LigneCommande d  ON(b.numCommande = d.numCmde)
LEFT OUTER JOIN  Article c ON(d.numArticle = c.numArticle)
LEFT OUTER JOIN Refus e  ON(d.numCmde = e.numCmde
AND d.numArticle = e.numArticle);
```

LMD : **Les jointures**

Jointure externe

Résultat Affichage trié (ORDER BY a.numClient ,b.numCommande **):**

Résultat de requête

SQL | Toutes les lignes extraites : 13 en 0,014 secondes

	NUMCLIENT	NOMCLIENT	NUMCOMMANDE	NUMARTICLE	DESIGNATION	QTECDEE	PRIX_TTC	QTEREFUSEE
1	101	nomCLient1	1	11	Clavier Gamer	1	539,968	-
2	101	nomCLient1	4	12	Souris SF	1	177	-
3	102	nomCLient2	2	12	Souris SF	2	177	-
4	102	nomCLient2	6	12	Souris SF	1	177	-
5	103	nomCLient3	3	13	Galaxy S50	1	354	-
6	103	nomCLient3	3	12	Souris SF	2	177	-
7	103	nomCLient3	3	14	iPhone XS	1	472	-
8	103	nomCLient3	3	11	Clavier Gamer	2	539,968	1
9	103	nomCLient3	5	11	Clavier Gamer	3	539,968	2
10	103	nomCLient3	5	12	Souris SF	4	177	-
11	104	nomCLient4	-	-	-	-	-	-
12	105	nomCLient5	-	-	-	-	-	-
13	106	nomClient6	-	-	-	-	-	-

LMD : **Le groupement**
GROUP BY ….. HAVING….

Le groupement, c'est effectuer un calcul en utilisant une ou plusieurs fonctions d'agrégations sur des groupes de lignes de données formés selon un ou plusieurs critères.

Syntaxe :

```
SELECT nomcol1, nomCol2 ,COUNT(ncol) ,MIN(ncol) ,MAX(ncol) ,
       SUM(ncol),  AVG(ncol)

FROM    NomTable1,..., NomTableN

WHERE  condition(s) de jointure

GROUP BY nomcol1, nomCol2

HAVING  COUNT(ncol) < …. (condition(s) sur les groupes formés ) ;
```

LMD : **Le groupement**
GROUP BY ….. HAVING….

Exemple :

Calculer et afficher le nombre de commandes par client.

```
SELECT numClient, COUNT(numCommande)
FROM Commande
GROUP BY numClient;
```

Exemple :

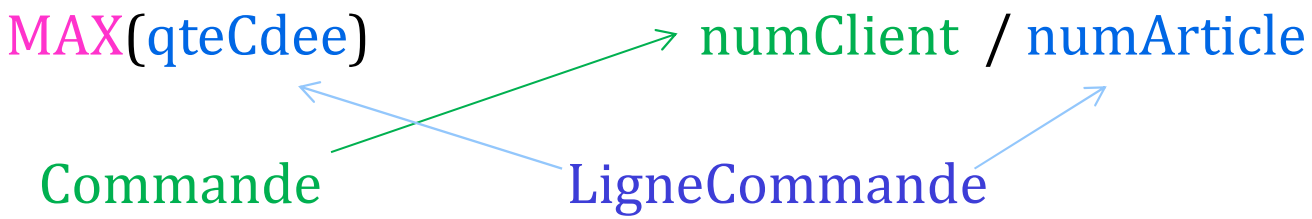
Afficher les clients qui ont passé un nombre de commandes supérieur à un.

```
SELECT a.*
FROM Client a, (SELECT numClient, COUNT(numCommande)
                FROM Commande
                GROUP BY numClient
                HAVING COUNT(numCommande)>1 ) b
WHERE a.numClient=b.numClient;
```

LMD : **Le groupement**
GROUP BY ….. HAVING….

Exemple :

Calculer et afficher les quantités maximales commandées par client et par article.




```
SELECT c.numClient,l.numArticle, MAX(l.qteCdee)
FROM Commande c, LigneCommande l
WHERE c.numCommande=l.numCmde
GROUP BY c.numClient,l.numArticle;
```

LMD : **Le groupement**
GROUP BY ….. HAVING….

Explication : formation des groupes de lignes de données par niveau.

Premier niveau : par numClient

Deuxième niveau : par numArticle

NUMCLIENT	NUMARTICLE	QTECDEE	NUMCOMMANDE
101	12	1	4
101	11	1	1
102	12	2	2
102	12	1	6
103	14	1	3
103	13	1	3
103	12	2	3
103	12	4	5
103	11	3	5
103	11	2	3

NUMCLIENT	NUMARTICLE	QTECDEE	NUMCOMMANDE
101	12	1	4
101	11	1	1
102	12	2	2
102	12	1	6
103	14	1	3
103	13	1	3
103	12	2	3
103	12	4	5
103	11	3	5
103	11	2	3

NUMCLIENT	NUMARTICLE	QTECDEE	NUMCOMMANDE
101	12	1	4
101	11	1	1
102	12	2	2
102	12	1	6
103	14	1	3
103	13	1	3
103	12	2	3
103	12	4	5
103	11	3	5
103	11	2	3

Résultat de la requête

NUMCLIENT	NUMARTICLE	MAX(L.QTECDEE)
101	11	1
101	12	1
102	12	2
103	11	3
103	12	4
103	13	1
103	14	1

LMD : **Le groupement**
GROUP BY ….. HAVING….

Si on inverse l'ordre des niveaux de sélection des groupes.

Premier niveau : par numArticle Deuxième niveau : par numClient

NUMARTICLE	NUMCLIENT	QTECDEE
11	101	1
11	103	2
11	103	3
12	101	1
12	102	2
12	102	1
12	103	2
12	103	4
13	103	1
14	103	1

NUMARTICLE	NUMCLIENT	QTECDEE
11	101	1
11	103	2
11	103	3
12	101	1
12	102	2
12	102	1
12	103	2
12	103	4
13	103	1
14	103	1

NUMARTICLE	NUMCLIENT	QTECDEE
11	101	1
11	103	2
11	103	3
12	101	1
12	102	2
12	102	1
12	103	2
12	103	4
13	103	1
14	103	1

NUMARTICLE	NUMCLIENT	MAX(L.QTECDEE)
11	101	1
11	103	3
12	101	1
12	102	2
12	103	4
13	103	1
14	103	1

Résultat de : ……

GROUP BY l.numArticle, c.numClient ;

Découpage différent

Schéma relationnel :

On reprend le même schéma relationnel :

Client (numClient, nomClient, prenomClient, dat_naiss, adresse)
Commande (numCommande, datCde, #numClient)
Article (numArticle, designation, couleur, prixHT)
LigneCommande(#numCmde,#numArticle, qteCdee)
Refus (#numCmde,#numArticle, datRefus, qteRefusee, motif)

Requêtes en SQL :

a. Afficher pour chaque client le montant total dépensé pour toutes ses commandes.
b. Afficher pour chaque client le montant total dépensé par commande.
c. Afficher pour chaque commande le nombre d'articles commandés et le nombre d'articles refusés.
d. Afficher pour chaque client, le nombre d'articles différents qu'il a commandé.

```sql
-- Afficher pour chaque client, le montant total qu'il a dépensé
-- sur toutes les commandes.
SELECT c.numClient,SUM(a.qtecdee * b.prixHT) AS montant_Total_HT
FROM ligneCommande a, Article b, Commande c
WHERE a.numArticle=b.numArticle
AND a.numcmde=c.numcommande
GROUP BY c.numClient
ORDER BY  c.numClient ASC;
```

Résultat de requête

SQL | Toutes les lignes extraites : 3 en 0,042 secondes

	NUMCLIENT	MONTANT_TOTAL_HT
1	101	607,6
2	102	450
3	103	3888

```sql
-- Afficher pour chaque client, le montant total qu'il a dépensé par commande.
SELECT c.numClient,a.numcmde, SUM(a.qtecdee * b.prixHT) AS montant_Total_HT
FROM ligneCommande a, Article b, Commande c
WHERE a.numArticle=b.numArticle
AND a.numcmde=c.numcommande
GROUP BY c.numClient,a.numcmde
ORDER BY c.numClient ASC ;
```

Résultat de requête

SQL | Toutes les lignes extraites : 6 en 0,013 secondes

	NUMCLIENT	NUMCMDE	MONTANT_TOTAL_HT
1	101	1	457,6
2	101	4	150
3	102	2	300
4	102	6	150
5	103	3	1915,2
6	103	5	1972,8

```
--Afficher pour chaque commande le nombre d'articles commandés
-- et le nombre d'articles refusés.
SELECT d.numCmde,COUNT(d.numArticle),COUNT(e.numArticle)
FROM  LigneCommande d , Refus e
WHERE d.numCmde = e.numCmde(+)
AND d.numArticle = e.numArticle(+)
GROUP BY d.numCmde ;
```

Résultat de requête

SQL | Toutes les lignes extraites : 6 en 0,003 secondes

	NUMCMDE	COUNT(D.NUMARTICLE)	COUNT(E.NUMARTICLE)
1	1	1	0
2	2	1	0
3	3	4	1
4	4	1	0
5	5	2	1
6	6	1	0

```
-- Afficher pour chaque client,le nombre d'articles différents qu'il a commandé.
SELECT c.numClient, COUNT(DISTINCT a.numArticle)AS NbreArticles
FROM ligneCommande a, Commande c
WHERE a.numcmde=c.numcommande
GROUP BY c.numClient ;
```

Résultat de requête

SQL | Toutes les lignes extraites : 3 en 0,002 secondes

	NUMCLIENT	NBREARTICLES
1	101	2
2	102	1
3	103	4

Les sous-requêtes

Au niveau de la clause SELECT

Exemple :

Calculer et afficher la proportion des clients mineurs.

Nbre de clients mineurs / Nbre total de clients * 100

↑ première sous-requête ↑ deuxième sous-requête

```
SELECT (SELECT COUNT(*)
         FROM Client
         WHERE ((SYSDATE-dat_Naiss)/365)<18)
        /
        (SELECT COUNT(*)
         FROM Client )
        * 100 AS "proportion"
FROM DUAL;
```

Remarque : DUAL : pseudo table qu'oracle met à disposition pour interroger le système. Dans ce cas on s'en sert pour afficher le résultat du calcul.

Les sous-requêtes

Au niveau de la clause FROM

Exemple :

Afficher les prénoms, noms et adresse des clients qui ont passé plus d'une commande.

Rque :Lorsque une sous-requête est utilisée au niveau d'une clause FROM, elle doit être nommée par un alias, ici b, pour pouvoir accéder aux colonnes qu'elle renvoie.

```
SELECT    a.prenomClient, a.nomClient, a.adresse


FROM Client a  ,  ( SELECT numClient, COUNT(numCommande)
                    FROM Commande
                    GROUP  BY numClient
                         HAVING COUNT(numCommande) > 1 ) b
  WHERE     a.numClient = b.numClient  ;
```

Les sous-requêtes

Au niveau de la clause WHERE

Exemple (Sous-requête non synchrone) :

Afficher l'article le plus cher : dont le prix correspond au prix le plus élevé.

```
SELECT    *
FROM Article
WHERE prixHT = ( SELECT MAX(prixHT)
                 FROM Article  )  ;
```

sous-requête qui renvoie une valeur

Ou bien

```
SELECT    *
FROM Article
WHERE prixHT >= ALL ( SELECT prixHT
                      FROM Article  )  ;
```

ON rajoute l'opérateur ALL car la sous-requête renvoie plusieurs valeurs.
ALL compare avec AND entre tous les éléments renvoyés par la sous-requête.

Les sous-requêtes

Au niveau de la clause WHERE

Lorsque la sous-requête renvoie plusieurs valeurs, on rajoute l'opérateur ALL ou ANY à la suite de l'un des opérateurs de comparaison usuels.

ANY compare avec OR entre tous les éléments renvoyés par la sous-requête.

Exemple :

Afficher les articles bleus dont le prix est égal au prix de l'un des articles rouges .

```
SELECT    *
FROM Article
WHERE couleur = 'bleu'
AND prixHT = ANY  ( SELECT prixHT
                    FROM Article
                    WHERE couleur = 'rouge')  ;
```

Les sous-requêtes

L'opérateur existentiel : EXISTS / NOT EXISTS

(Sous-requête synchrone)

L'opérateur existentiel est utilisé avec les sous-requêtes pour tester si elles renvoient ou ne renvoient pas des données.

Syntaxe :

```
SELECT .....
FROM .....
WHERE EXISTS ( SELECT *
               FROM ......
               WHERE conditions ) ;
```

Renvoie VRAI si la sous-requête qui le suit renvoie des données. Plus exactement, à la première condition vérifiée vraie de la sous-requête.

Renvoie FAUX si la sous-requête qui le suit ne renvoie aucune donnée. Autrement dit, lorsque toutes des conditions de la sous-requête sont évaluées à Faux.

Les sous-requêtes

L'opérateur existentiel : EXISTS / NOT EXISTS

Exemple 1 :

Afficher les articles qui ont été commandés.

```
SELECT    *

FROM Article  a

WHERE EXISTS ( SELECT *

                FROM LigneCommande b

                WHERE a.numArticle = b.numArticle  )  ;
```

Exemple 2 :

Afficher les articles qui n'ont jamais été commandés.

```
SELECT    *

FROM Article  a

WHERE NOT EXISTS ( SELECT *

                    FROM LigneCommande b

                    WHERE a.numArticle = b.numArticle  )  ;
```

Les sous-requêtes

L'opérateur existentiel : EXISTS / NOT EXISTS

Exemple 3 :

Afficher les commandes sur lesquelles figurent tous les articles. Autrement dit, les commandes où il n'y a pas un article qui n'y figure pas.

```
SELECT *
FROM Commande a
WHERE NOT EXISTS (SELECT     *
                  FROM Article  b
                  WHERE NOT EXISTS ( SELECT *
                                     FROM LigneCommande c
                                     WHERE a.numCommande = c.numCmde
                                     AND   b.numArticle  = c.numArticle  ))  ;
```

Exemple 3 (bis) :

Afficher les commandes sur lesquelles figurent tous les articles.
Autrement dit, les commandes ayant un nombre d'articles égal au nombre total d'articles.
On va y répondre en utilisant la clause GROUP BY.

```
SELECT *
FROM Commande a ,( SELECT numCmde, COUNT(numArticle)
                   FROM LigneCommande
                   GROUP BY numCmde
                   HAVING  COUNT(numArticle) = ( SELECT COUNT(*)
                                                 FROM Article) ) b
WHERE a.numCommande = b.numCmde  ;
```

Les Vues : VIEW

Syntaxe :

CREATE OR REPLACE VIEW NomVue (nomCol1Vue, …., nomColNVue)
AS
SELECT nomCol1, …., nomColN
FROM ….
WHERE …… **;**

Doivent correspondre en nombre et en ordre

```
SELECT   *
FROM NomVue ;        -- Affiche ce que renvoie la vue.
```

Une vue c'est une requête enregistrée sous un nom **NomVue**. Ce qui est stocké en mémoire c'est la requête que définit cette vue.

Lorsque le système rencontre une vue au niveau d'une clause FROM, il exécute la requête correspondante et les données qui sont renvoyées, par cette dernière, vont servir à la sélection dans la requête principale.

Les Vues : VIEW

Syntaxe :

CREATE OR REPLACE VIEW NomVue
AS
SELECT nomCol1, …., nomColN
FROM ….
WHERE …… **;**

Si on ne spécifie pas des noms de colonnes pour la vue, cette dernière prendra par défaut les noms de colonnes citées au niveau du SELECT.

Pourquoi **OR REPLACE** : la syntaxe de la requête peut être mise à jour, au lieu de supprimer la vue et la redéfinir avec la nouvelle syntaxe, un remplacement sera effectué.

Supprimer une vue : **DROP VIEW NomVue**;

Afficher la structure d'une vue : **DESCRIBE NomVue**;

Les Vues : VIEW

Exemple : Création de vue InfoCommande

```
CREATE OR REPLACE VIEW  InfoCommande ( numero_Client, nom, numero_Commande,
numero_Article, designation, quantiteeCommandee, prix_unitaire_TTC, quantiteeRefusee)
AS SELECT a.numClient, a.nomClient, b.numCommande,
d.numArticle, c.designation, d.qteCdee, prixHT*1.18, qteRefusee
FROM Client a, Commande b, Article c, LigneCommande d , Refus e
WHERE a.numClient = b.numClient(+)
AND b.numCommande = d.numCmde(+)
AND d.numArticle = c.numArticle(+)
AND d.numCmde = e.numCmde(+)
AND d.numArticle = e.numArticle(+)
ORDER BY a.numClient ASC;
```

Sortie de script × | Résultat de requête ×

Tâche terminée en 0,051 secondes

```
View INFOCOMMANDE créé(e).
```

Les Vues : VIEW

Affichage de la vue InfoCommande :

```
SELECT * FROM InfoCommande;
```

Sortie de script | Résultat de requête

SQL | Toutes les lignes extraites : 13 en 0,013 secondes

	NUMERO_CLIENT	NOM	NUMERO_COMMANDE	NUMERO_ARTICLE	DESIGNATION	QUANTITEECOMMANDEE	PRIX_UNITAIRE_TTC	QUANTITEEREFUSEE
1	101	nomCLient1	4	12	Souris SF	1	177	-
2	101	nomCLient1	1	11	Clavier Gamer	1	539,968	-
3	102	nomCLient2	2	12	Souris SF	2	177	-
4	102	nomCLient2	6	12	Souris SF	1	177	-
5	103	nomCLient3	3	12	Souris SF	2	177	-
6	103	nomCLient3	5	12	Souris SF	4	177	-
7	103	nomCLient3	5	11	Clavier Gamer	3	539,968	2
8	103	nomCLient3	3	14	iPhone XS	1	472	-
9	103	nomCLient3	3	11	Clavier Gamer	2	539,968	1
10	103	nomCLient3	3	13	Galaxy S50	1	354	-
11	104	nomCLient4	-	-	-	-	-	-
12	105	nomCLient5	-	-	-	-	-	-
13	106	nomClient6	-	-	-	-	-	-